```
  createSubscription(source: SoldOutDataController): Subscription {
    return createConsumer().subscriptions.create("ScheduleChannel", {
      received({ soldOutConcertIds }) {
        source.updateData(soldOutConcertIds)
      },
    })
  }

  updateData(soldOutConcertIds: number[]): void {
    this.concertTargets.forEach((concertElement: HTMLElement) => {
      concertElement.dataset.concertSoldOutValue = soldOutConcertIds
        .includes(parseInt(concertElement.dataset.concertIdValue, 10))
        .toString()
    })
  }
}
```

This class has changed somewhat to support receiving from ActionCable. In the first line, we import `createConsumer` and `Subscription` from the `@rails/actioncable` library. In our Stimulus `connect` method, we check to see if the subscription has been created, and if not, we create it.

The `createSubscription` method is what actually connects our client to the server via a WebSocket. We call the method `createConsumer().subscriptions.create`. The first part of this `createConsumer()` returns a `Cable` class, which knows about subscriptions, and we can call the `create` method to create a new subscription. The call to `create` takes two arguments. The first is the name of the channel class on the server (not the name of the stream). The second argument is a JavaScript object that defines methods that are automatically called when various events happen. The method we are concerned with is `received`, which is called when new data is received. The other two that you can have here are `connected` and `disconnected`.

The `received` method is a callback invoked when data is passed, and it's doing the same thing with the data that we were doing before, just in a slightly different order. It decodes the data and passes it to `updateData`, which does the same Stimulus dataset updates as this code did before. We have a couple of type issues, and decoding this way keeps the concert IDs as integers rather than strings, so we need to do an integer comparison rather than a string.